

ARQUITECTURA DE SOLUCIONES

SISTEMA DE BANCARIO

JUNIO 2023

1. Tabla de Contenidos

1. Tabla de Contenidos.....	1
2. Descripción general del sistema.....	2
3. Decisiones Arquitectónicas y justificaciones.....	4
4. Diagrama C4 - Contexto.....	6
5. Diagrama C4 - Contenedores.....	7
6. Diagrama C4 - Componentes.....	7
7. Flujo de Autenticación.....	12
8. Flujo de Onboarding Biométrico.....	12
9. Arquitectura de Auditoría y Logs.....	12
10. Estrategia de Caché (Redis).....	13
11. Arquitectura de Notificaciones.....	13
12. Normativa Bancaria.....	13
13. Monitoreo.....	14
14. Conclusiones.....	14

2. Descripción general del sistema

El presente documento va a describir la arquitectura de soluciones propuesta y diseñada para un sistema de Banca Web y Móvil. El sistema permitirá a los usuarios que accedan a cualquier plataforma, verificar su historial de movimientos, realizar transferencias ya sea entre cuentas propias, interbancarios (SPI) o directos interbancarios (PAD). La arquitectura propuesta se realizó bajo el diseño del modelo C4, abarcando los niveles de Contexto, Contenedores y Complementos.

La solución está construida sobre una arquitectura de microservicios para que sea escalable a largo y corto plazo e independiente. Fue organizada en tres capas principales: Backend for Frontend, Integrador y Core, las cuales se expondrán a través de un API Gateway centralizado.

Estas plataformas se pueden desplegar en Microsoft Azure así como en un servidor local.

A continuación una breve descripción del análisis desarrollado para la implementación del sistema:

Actores principales:

- Usuarios
- Administrador

Sistemas Internos:

Frontends:

- Aplicación Web (SPA) - Angular
- Aplicación Móvil - Flutter.

Backend / Plataforma bancaria:

- API Gateway. (.Net 10)
- Microservicios de autenticación, movimientos, transferencias, notificaciones, monitoreo. (.Net 10)

Bases de datos:

- Autenticación.
- Movimientos.
- Transferencias (propias, interbancarias, PAD).
- Notificaciones.
- Auditoría y logs.
- Monitoreo.

- Redis

Capa BFF

- BFF.Autenticación
- BFF.Movimientos
- BFF.Transferencias
- BFF.Notificación
- BFF.Onboarding

Capa Integrador

- Integrador.Autenticación
- Integrador.Movimientos
- Integrador.Transferencias
- Integrador.Notificación
- Integrador.Onboarding
- Integrador.Monitoreo
- Integrador.Logs

Capa Core

- Core.Autenticación
- Core.Movimientos
- Core.Transferencias
- Core.TransferenciasSPI
- Core.TransferenciasPAD
- Core.Notificación
- Core.Onboarding
- Core.Monitoreo
- Core.Logs

Bases de datos:

- Autenticación
- Movimientos
- Transferencias
- Notificación
- Onboarding
- Monitoreo
- Logs
- Redis

Servicios externos:

- Swift Bridge
- BanRed
- Banco Central Ecuador
- Prometheus /Grafana

- Azure Face API
- Azure Communication Services

Capa	Tecnología
BackEnd	.Net 10
FrontEnd	Angular
Móvil	Flutter
API Gateway	Azure API Management
Mensajería	RabbitMQ
Cache	Redis
Autenticación	Azure AD B2C (OAuth 2.0)
Biométrico	Azure Face API
Monitoreo	Azure Monitor + Grafana
Cloud	Azure
Secretos	Azure Key Vault
Base de Datos	SQL Server

3. Decisiones Arquitectónicas y justificaciones

Se realizó un análisis para la arquitectura del sistema las cuales se consideraron alternativas varias. En donde se realizaron justificaciones técnicas o de negocios.

A continuación se detalla las decisiones tomadas para la implementación del diseño de arquitectura:

Arquitectura	Justificación
API Gateway (Azure API Management / .NET 10)	• Seguridad Centralizada esto quiere decir concentración de las principales funcionalidades, como autenticación, autorización, validación de claims, así como otras funcionalidades.

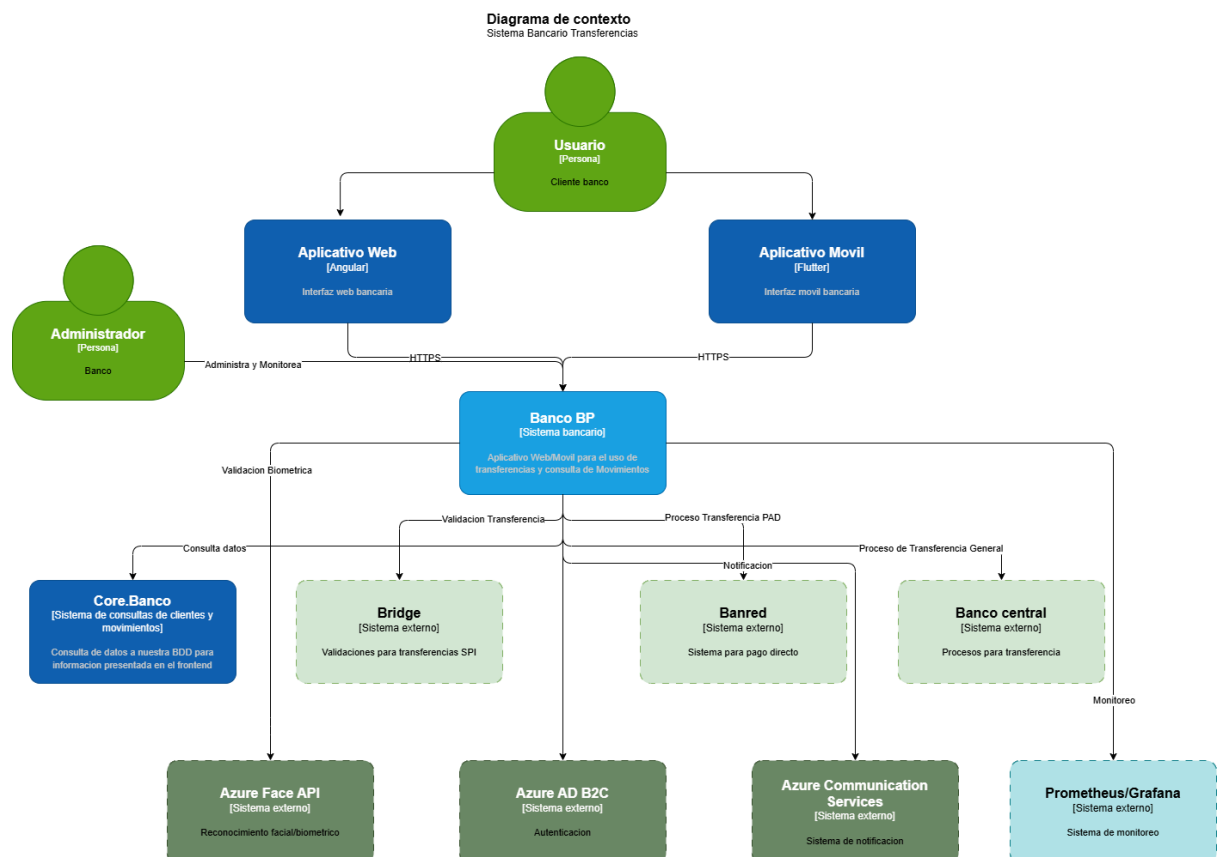
	• Escalabilidad y desacoplamiento permite añadir nuevos microservicios con nuevas funcionalidades sin afectar al cliente final por la complejidad que estos pueden tener.
Microservicios en .NET 10 (Minimal API)	• Escalabilidad, desacoplamiento y mantenibilidad para que cada dominio pueda escalar sin afectar a los demás. • Rendimiento y modernidad para ofrecer endpoints ligeros y rápidos, optimizando consumo de recursos y tiempo de respuesta.
Base de datos separadas por dominio	• Escalabilidad y resiliencia para que cada base pueda crecer y replicarse de forma independiente. • Seguridad y cumplimiento normativo permite separar datos sensibles aplicando controles específicos para así cumplir con la regularización de protección de datos
Frontend (Angular + Flutter)	• Experiencia de usuario con angular tenemos una robusta web ofrecidas por SPA, mientras que Flutter permite aplicaciones multiplataformas con alto rendimiento y UI nativa • Productividad y comunidad estas dos tecnologías permiten reducir riesgos y son de fácil mantenimiento.

Patrón	Aplicación en el sistema
Repository Pattern	Abstrae el acceso a datos cada Core accede a su Base de Datos a través de repositorios.
Adapter Pattern	Usado en los integradores para

	conectarse al Core Bancario y sistemas complementarios.
Observer / Publisher	Los Core publican eventos a RabbitMQ, Core.
Cache - Aside	Redis se consulta antes de ir a SQL Server.
Strategy Pattern	Define una familia de algoritmos, encapsular cada uno y hacerlos intercambiables en tiempo de ejecución.
Event Sourcing (Logs)	Cada acción genera un evento inmutable en Base de Datos.
Factory Pattern	Centraliza y encapsula la creación de objetos complejos evitando que el código de negocios dependa directamente de clases.
CQRS	Separación de lectura y escritura

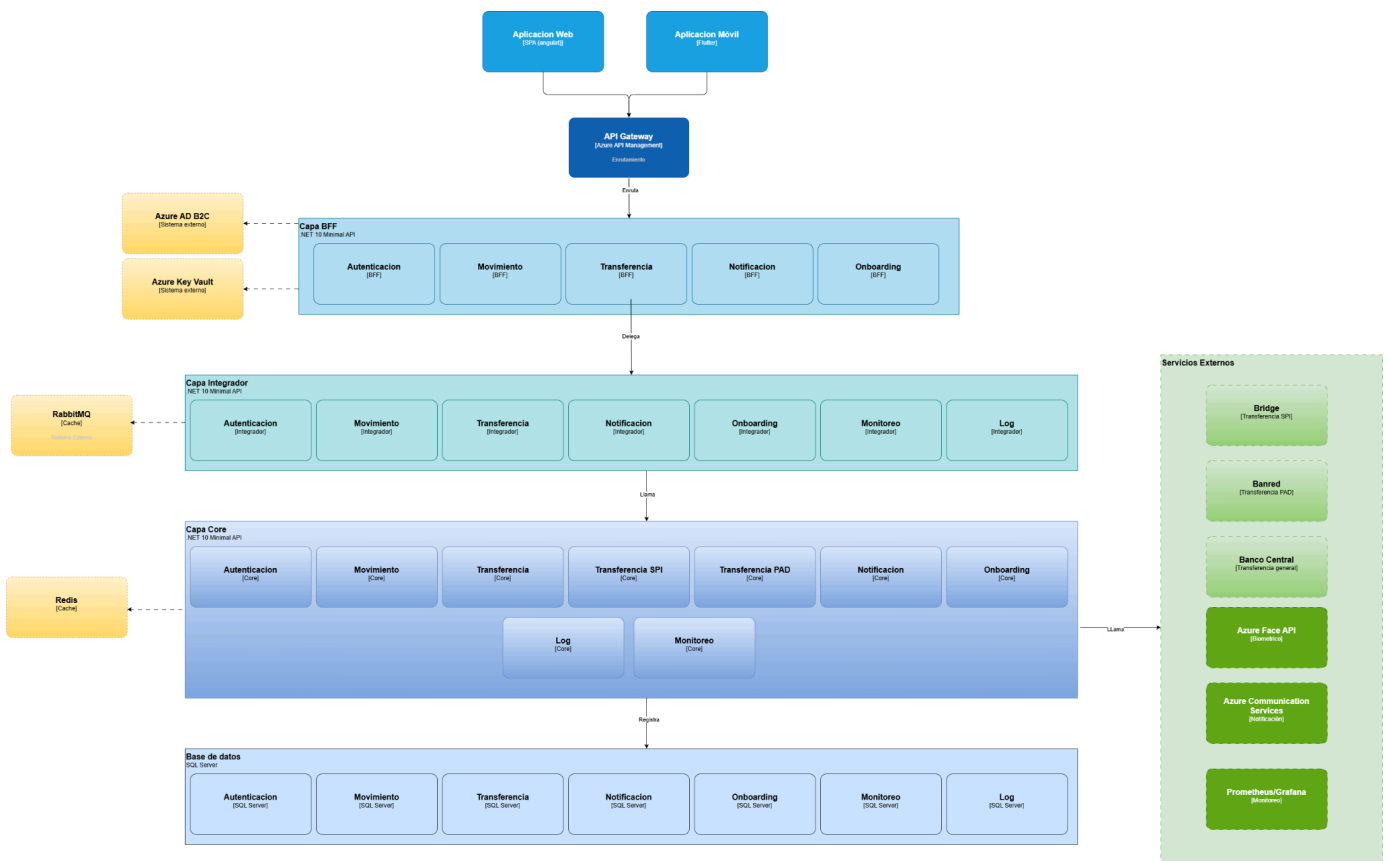
4. Diagrama C4 - Contexto

Este diagrama tiene como fin mostrar los actores que van a interactuar en el sistema, y los sistemas externos con los que se va a comunicar.



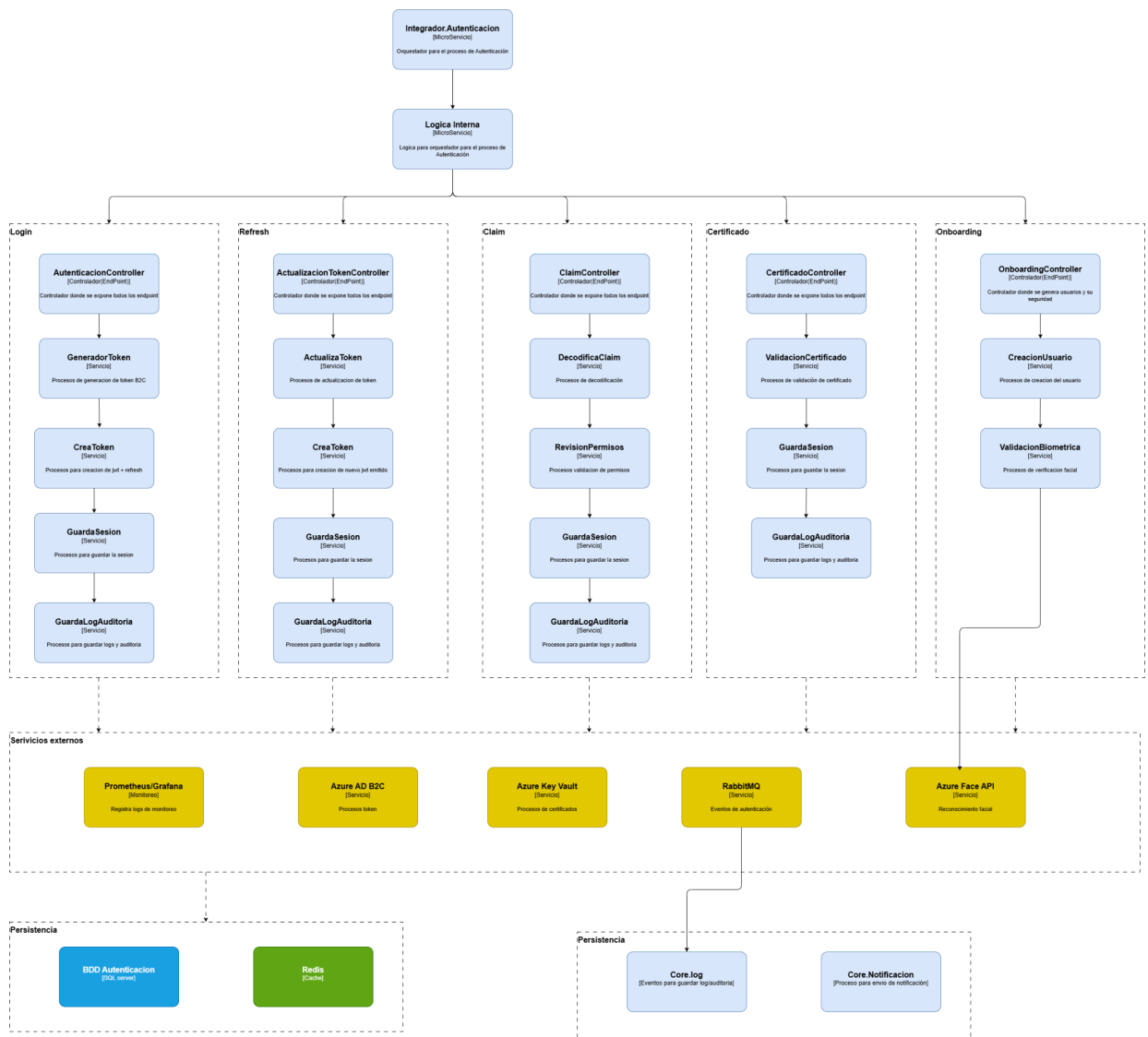
5. Diagrama C4 - Contenedores

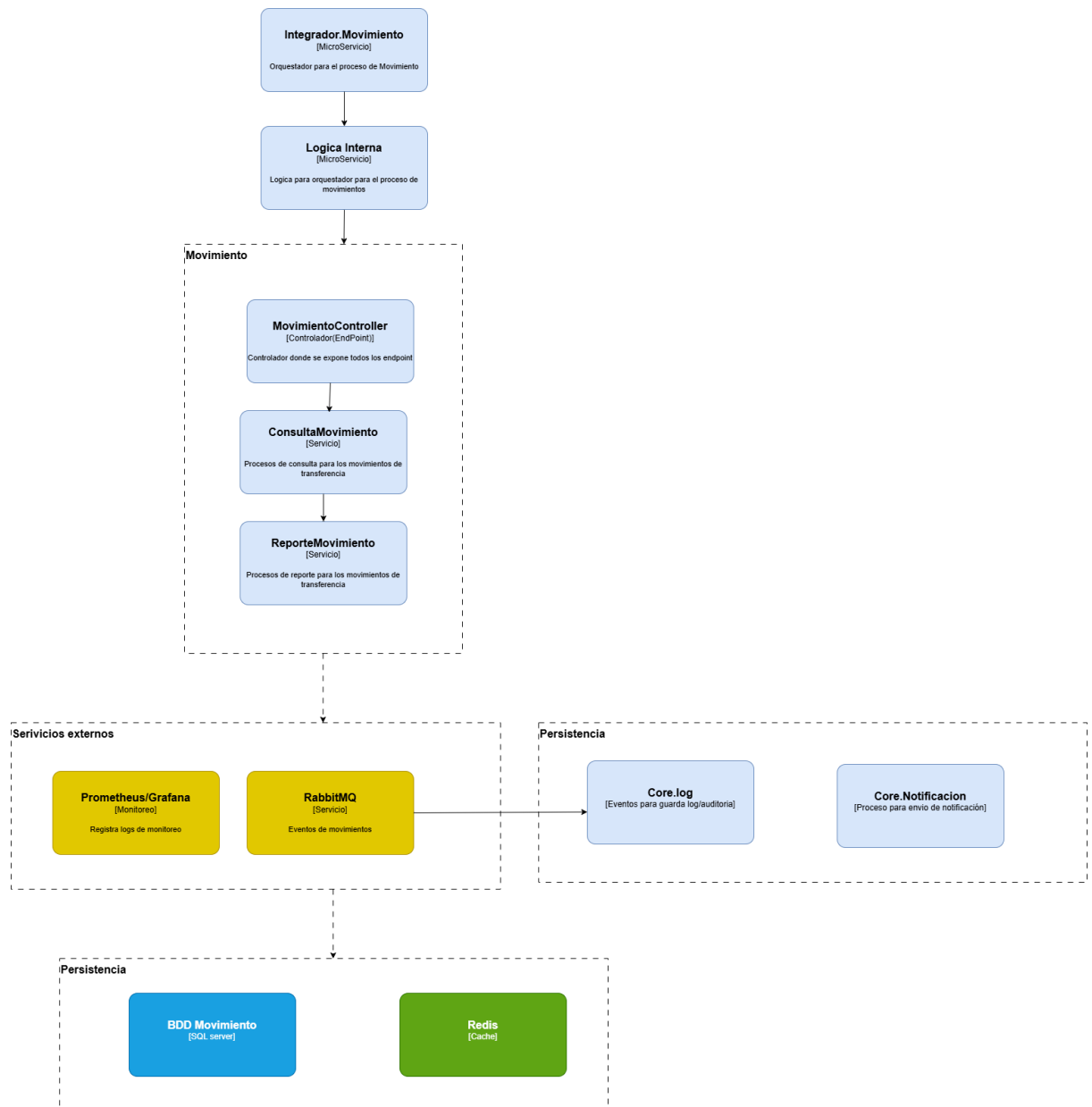
Este tipo de diagrama muestra cómo están comunicados los microservicios, bases de datos, frontends y cache y los sistemas externos.

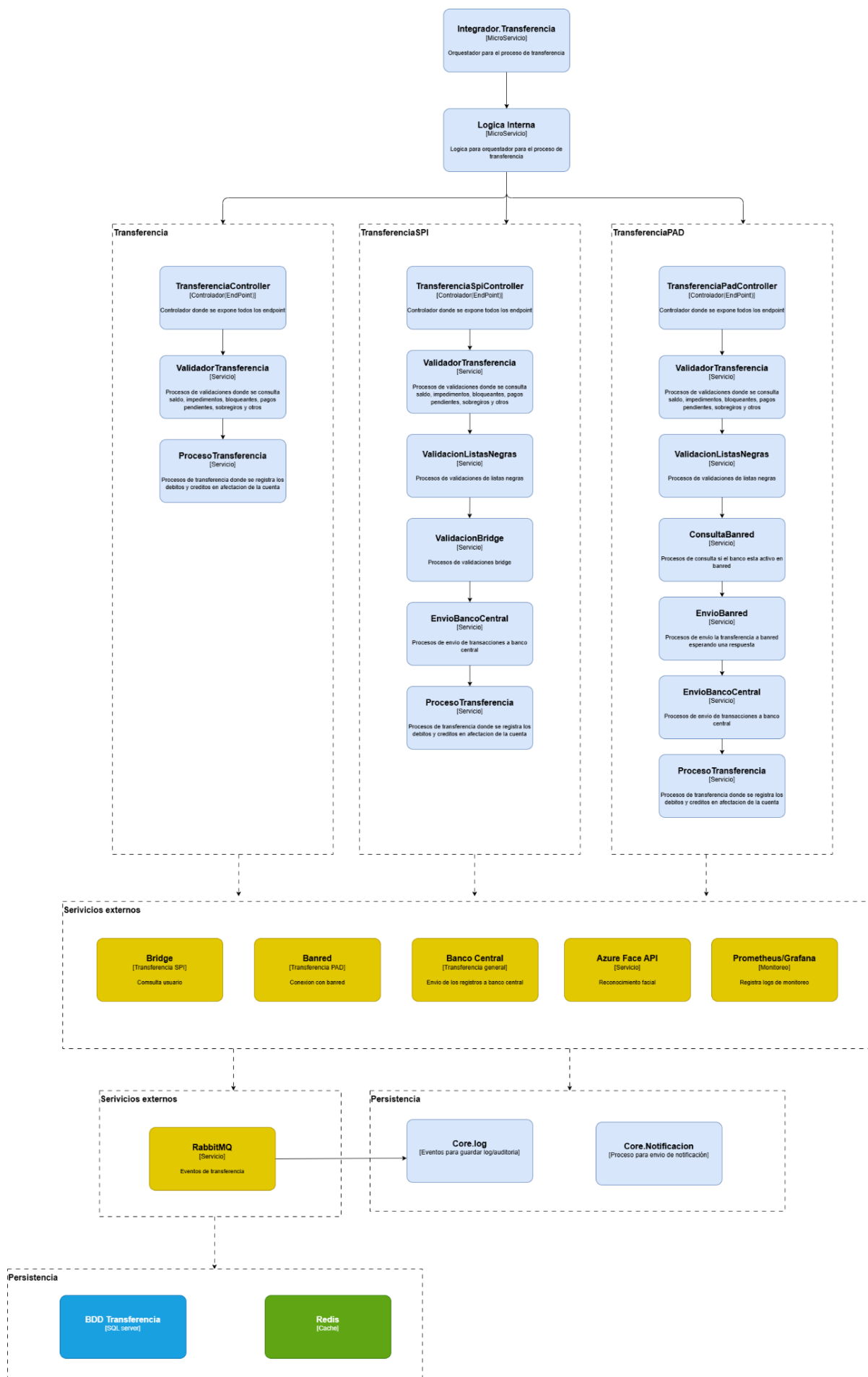


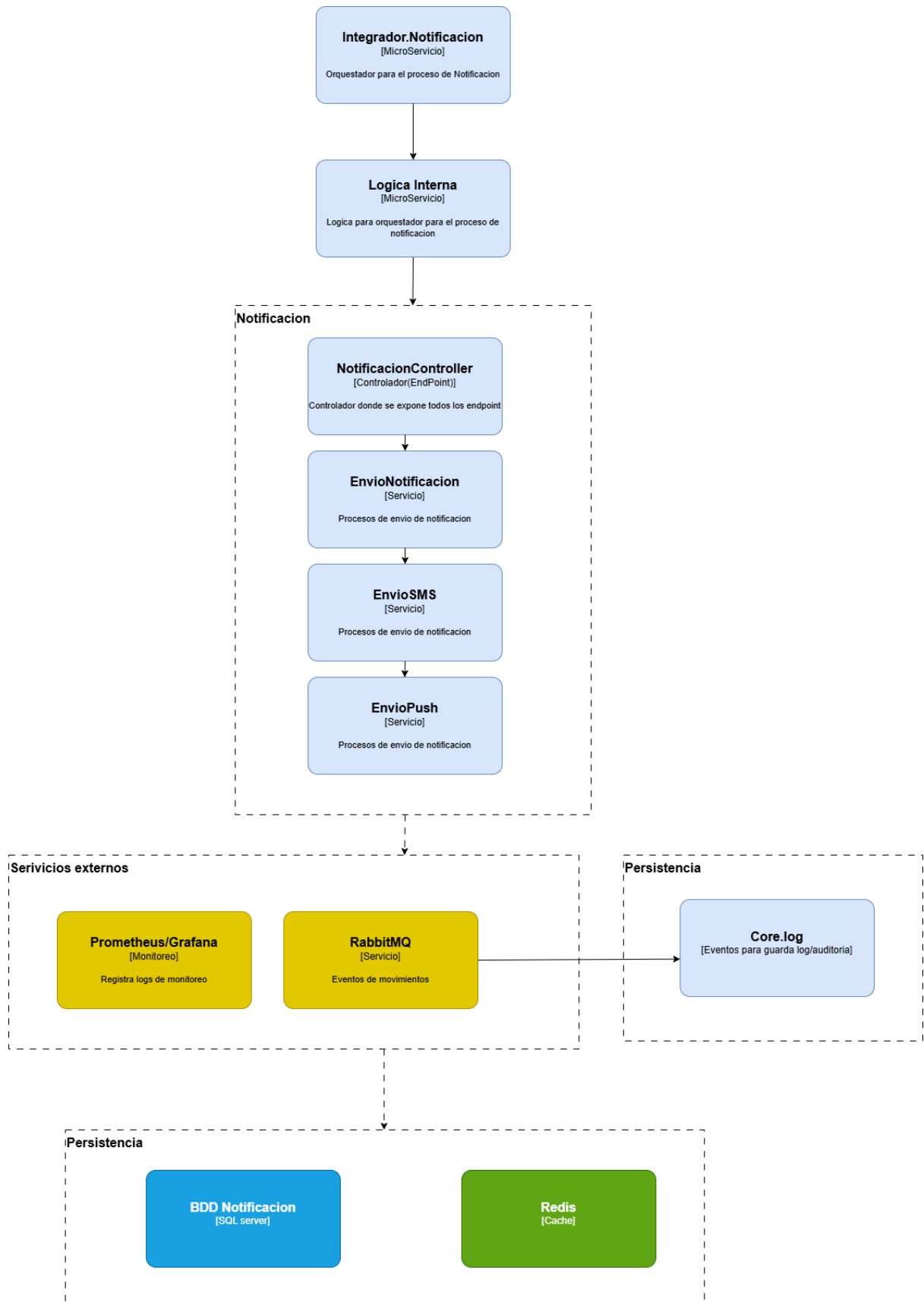
6. Diagrama C4 - Componentes

En este diagrama se muestra la estructura interna de microservicios más relevantes









7. Flujo de Autenticación

El sistema nos va a permitir implementar el sistema OAuth 2.0 con el flujo de autorización, recomendado para la aplicación móvil y SP donde es posible guardar los secret de forma segura.

- La aplicación SPA genera un código aleatorio y su algoritmo criptográfico.
- Redirige al usuario a Azure AD B2C con el algoritmo criptográfico.
- Cuando el usuario ingresa las credenciales, Azure AD B2C autentica.
- Azure AD B2C retoma un código de autenticación de vida corta al cliente.
- La aplicación intercambia códigos para verificar el acceso con el token y refresca el token.
- Cada petición al API Gateway incluye el acceso a token
- El API Gateway valida el token con Azure AD B2C.
- Al expirar el token, la aplicación usa el refresco para obtener un nuevo token.

8. Flujo de Onboarding Biométrico

El proceso de onboarding permite registrar nuevos clientes en la aplicación móvil.

- El cliente descarga la aplicación Móvil y empieza el proceso
- Cliente completa el formulario biométrico y básico.
- La aplicación solicita los permisos requeridos para los datos biométricos.
- Azure Face API compara la foto con el documento de identidad
- Si la validación es exitosa se crea un usuario en Azure AD B2C.
- Se generan credenciales iniciales y se envía a notificaciones.
- El nuevo usuario puede ingresar al sistema, con los datos biométricos ingresados.
- Todo el proceso queda registrado en Logs y Auditoría.

9. Arquitectura de Auditoría y Logs

Se implementa un mecanismo de auditoría completo e inmutable para todas las operaciones realizadas por los clientes y los procesos internos.

Elemento	Descripción
Evento Generador	Cada microservicio genera un evento al finalizar cualquier operación
Async	El evento se publica a RabbitMQ sin bloquear la operación principal
Consumidor	Core. logs consume la mensajería de cola
Campo del evento	Servicio, usuario, acción, timestamp, IP, payload, correlationId.
Trazabilidad	El correlationId se propaga en todas las capas para la trazabilidad.

10. Estrategia de Caché (Redis)

El sistema utiliza Redis como capa de cache distribuida compartida entre todos los microservicios, esto garantiza consistencia entre la cache y la base de datos principal.

11. Arquitectura de Notificaciones

El sistema implementa canales de notificación para cumplir los requerimientos de normativas bancarias. Eso se gestionará a través de Azure Communication Services.

12. Normativa Bancaria

El sistema está diseñado para cumplir con las principales normativas aplicables a entidades financieras en el Ecuador y estándares internacionales de seguridad bancaria.

Control	Implementación
Ley Organiza de datos personales	Consentimiento explícito para el uso de datos biométricos.
Resolución Junta Bancaria del Ecuador	Cumplimiento de la normativa de la Superintendencia de Bancos,

	autenticación robusta y registro de logs de auditoría.
ISO/IEC 27001	Políticas de seguridad de la información, gestión de accesos, control de cambios y gestión de inicio de sesión 1
OWASP	Protección contra las principales vulnerabilidades web
Cifrado en tránsito	TLS 1.2 obligatorio en todas las comunicaciones.
Gestión de secretos	Azure Key Vault para connection strings.
Token de corta vida	Token de máximo 10 minutos con refresco

13. Monitoreo

El sistema implementa observabilidad completa mediante el stack Prometheus y Grafana, complementando con Azure Monitor y Application Insights para trazabilidad distribuida.

14. Conclusiones

La arquitectura propuesta para el sistema Bancario está diseñada para satisfacer todos los requerimientos funcionales y no funcionales del negocio, garantizando seguridad, escalabilidad, alta disponibilidad y cumplimiento normativo.

- Arquitectura de microservicio que garantiza desacoplamiento, autonomía de datos y facilidad de mantenimiento.
- Implementación completo de OAuth 2.0 con flujo Authorization Code
- Onboarding biométrico con Azure Face API integrado al flujo de autenticación, cumpliendo con los requerimientos de seguridad de identidad digital.
- Auditoría vía RabbitMQ garantizado trazabilidad completa de todas las operaciones bancarias.
- Cumplimiento de normativas bancarias, y ley orgánicas de datos personales de Ecuador-
- Canales de notificación vía Azure Communication Services cumpliendo el requerimiento regulatorio.